CVR-To-Summary Reconciliation

Gio Della-Libera

Draft — June 28, 2026

Abstract

Cast-vote records are only useful for public verification if their contest interpretations reconcile to the public summaries they support. This paper defines the RCOUNT CVR-to-summary contract: ballot-card or CVR-row counts must match manifest accounting, contest selections and residuals must sum to counted ballots, and aggregated CVR interpretations must match published summary totals. The current fixtures exercise that contract with `cvr_summary_total`: the positive fixture passes when CVR contest rows aggregate to the public summaries, while the negative fixture fails when a CVR row is changed without a matching summary change.

1 Introduction

Cast-vote records are the interpreted contest marks exported by a voting system. They can make public verification much stronger: instead of trusting only a summary row, an auditor can recompute contest totals from lower-level records. They can also make verification more dangerous if ballot-level detail exposes rare vote patterns, ballot styles, or small-cell combinations.

V.6 defines the reconciliation contract between CVR evidence and public summaries. The current RCOUNT crates now normalize a small `normalized/cvr.ndjson` surface: one row per CVR contest interpretation. Those rows aggregate to public summaries through `cvr_summary_total`, while `contest_selection_sum` remains the summary-side guardrail that checks selections, write-in buckets, undervotes, overvotes, and blank contests against declared counted ballots.

2 Reconciliation Contract

The full CVR-to-summary contract has three layers:

Layer	Required reconciliation
Manifest to CVR rows	Ballot-card or CVR-row counts match ballot manifest accounting for the represented batch or reporting unit.
CVR rows to contest aggregates	Candidate selections, write-in buckets, undervotes, overvotes, and blank contests aggregate by contest.
Contest aggregates to summaries	Aggregated CVR interpretations match published summaries by contest, reporting unit, status, and optional batch.

$$\begin{aligned}\texttt{manifest card count} &= \texttt{CVR row/card count} \\ &= \texttt{summary counted basis}\end{aligned}$$

$$\sum \texttt{CVR contest interpretations} = \texttt{summary selections and residuals}$$

The word CVR must stay precise. A CVR row is not necessarily a voter or a whole ballot; some systems export one row per ballot card, one row per sheet side, or one row per contest grouping. A CVR can record an undervote, overvote, blank contest, adjudicated write-in bucket, or contest not on that ballot style. V.6 therefore treats the adapter's row semantics as part of the evidence contract, not as an implementation detail.

The first normalized RCOUNT row uses this package-local contract:

Field	Meaning
`cvr_id`	Package-local CVR/card identifier; not a voter identifier.
`contest_id`	Contest interpreted by this row.
`reporting_unit_id`	Public reporting unit for aggregation.
`batch_id`	Optional batch link when manifest evidence is batch-level.
`status`	Count status represented by the row.
`selection_ids`	Candidate or write-in bucket selections interpreted for the contest.
`undervote`, `overvote`, `blank_contest`	Residual contest interpretation flags.
`source_refs`	Source evidence rows supporting the normalized CVR row.

The implemented verifier treats the row as one contest interpretation. Each CVR id and contest id pair must be unique. A row fails if it combines a candidate selection with a residual flag, sets multiple residual flags, references an unknown selection id, or contains more selections than the contest's `vote_for` rule permits.

Check	Failure caught
`cvr_summary_total`	CVR aggregates disagree with public summary selections or residuals.
`contest_selection_sum`	Summary selections plus residuals do not equal counted ballots.
`jurisdiction_contest_total`	Lower-level summaries do not roll up to the jurisdiction total.
`source_hash_match`	Published source evidence bytes no longer match declared hashes.

3 Current Equations

The current implementation exercises both the normalized CVR aggregate and the summary-side guardrail. The CVR equation is:

$$\sum \texttt{normalized CVR contest rows} = \texttt{summary selections and residuals}.$$

For each contest, reporting unit, status, and optional batch, the verifier aggregates `selection_ids`, `undervote`, `overvote`, and `blank_contest` fields from `normalized/cvr.ndjson`. It then compares those aggregates to `normalized/summaries.ndjson`. A mismatch fails as `cvr_summary_total`.

The summary guardrail remains `contest_selection_sum`:

$$\texttt{counted\_ballots} = \sum \texttt{selection votes} + \texttt{undervotes} + \texttt{overvotes} + \texttt{blank\_contests}.$$

RCOUNT also checks `jurisdiction_contest_total`, which rolls lower-level reporting-unit summaries up to the jurisdiction summary. Together, these checks catch arithmetic drift before a CVR adapter is allowed to claim that lower-level rows support the public totals.

V.6 implements the CVR-to-summary gate; V.5 supplies manifest controls

4 Fixtures

The positive fixture is `cvr-summary`. It contains 140 normalized CVR contest rows: 80 rows for P-001 and 60 rows for P-002. For precinct P-001, the public summary declares 80 counted ballots:

Unit	A	B	Write-in	Under	Over	Blank
P-001	40	35	1	3	1	0
P-002	25	30	0	4	0	1

For P-001, the CVR aggregate is $40 + 35 + 1 + 3 + 1 + 0 = 80$, so `cvr_summary_total` and `contest_selection_sum` both pass.

The negative fixture is `bad-cvr-summary`. It changes one P-001 CVR row from `cand-a` to `cand-b` while leaving the public summaries unchanged. The summary arithmetic still passes, but the CVR aggregate no longer matches the summary: Candidate A computes to 39 instead of 40.

Path	Role
`normalized/cvr.ndjson`	Normalized CVR contest interpretation rows.
`normalized/summaries.ndjson`	Public summary targets for CVR aggregation.
`normalized/contests.ndjson`	Selection ids and `vote_for` constraints.
`sources/source-index.json`	Raw source hash evidence.
`transcripts/verify-transcript.json`	Verifier pass/fail transcript.

The regression caught by `bad-cvr-summary` is precise: a verifier that checks only summary arithmetic, or silently ignores `normalized/cvr.ndjson`, would miss a CVR-side selection drift.

5 Verifier Traceability

The fixtures can be regenerated and checked directly:

```
cargo run -p rcount-io --example cvr_summary_package
cargo run -p rcount-io --example bad_cvr_summary_package
cargo run -p rcount-cli -- verify \
  docs/examples/rcount-golden-packages/cvr-summary
cargo run -p rcount-cli -- verify \
  docs/examples/rcount-golden-packages/bad-cvr-summary
```

Fixture	Expected result	Carrying checks
`cvr-summary`	transcript status `pass`	`cvr_summary_total`, `contest_selection_sum`, `jurisdiction_contest_total`, and `source_hash_match` pass.
`bad-cvr-summary`	transcript status `fail`	`cvr_summary_total` fails for P-001 while source hashes still pass.

The negative transcript anchors the failure:

```
{
  "equation_id": "cvr_summary_total",
  "status": "fail",
  "error": "CVR summary mismatch for contest syn-2024-mayor \
reporting unit syn:precinct:P-001 field selection:cand-a: summary 40, cvr 39"
}
```

6 Boundaries

V.6 does not say a voting system captured voter intent correctly. A CVR export is an interpretation artifact. Hand audits, risk-limiting audits, recounts, and adjudication records remain separate evidence.

V.6 also does not require every jurisdiction to publish ballot-level CVRs. Some jurisdictions do not produce them, some publish only redacted or transformed versions, and some contests are too small for safe ballot-level publication. RCOUNT can still verify summary and manifest layers when public CVRs are absent.

The implemented row shape is a normalized interchange surface, not a direct vendor schema. V.9 owns vendor CSV, JSON, XML, and NIST-style adapter details. Adapters must preserve enough raw source evidence for an independent parser to disagree with the normalized rows.

Finally, CVR publication must inherit V.4's receipt-safety rule. Rare ballot styles, write-ins, timestamps, scanner batches, or unique selection patterns can turn a CVR into identifying evidence. A deployable CVR layer needs redaction, aggregation, or restricted-publication policy in addition to arithmetic checks.

CVR publication pattern	Risk
Rare write-in plus tiny reporting unit	Narrows a ballot to a small group or one voter.
Unique cross-contest selection pattern	Can become a recognizable ballot signature.
Ballot style plus batch or scanner time	Can join with check-in or observation records.
Single-card provisional or cure batch	Can reveal whether a specific ballot was counted.
Joining CVRs to inclusion tokens	Can turn inclusion verification into a choice receipt.

Public CVR packages may therefore need aggregation, suppression, delayed publication, or restricted evidence views. Hashes make evidence tamper-evident; they do not make ballot-level records private.

7 Conclusion

CVR-to-summary reconciliation is where public verification moves from totals to interpreted ballot evidence. The implemented V.6 slice reconciles normalized CVR contest rows to public summaries and keeps the summary-side invariant in place: selections plus residuals must equal counted ballots, and bad local evidence must not hide inside jurisdiction totals.

That gives V.7 a stable base for replayable risk-limiting audits, while keeping the remaining CVR privacy and vendor-adapter questions visible instead of pretending they are already finished.

References

Gio Della-Libera. Rcount overview: Reproducible election count packages. Technical report, Apportionment Research, 2026a.

Gio Della-Libera. Rcount substrate specification. Technical report, Apportionment Research, 2026b.

Gio Della-Libera. Ballot manifest verification. Technical report, Apportionment Research, 2026c.